

Chapter 17



Attacking the Web Server:

Web applications rely on underlying components such as the web server, operating system, and network infrastructure. Attackers may target these layers, and compromising them can lead to a full compromise of the application. Web server vulnerabilities are particularly important because they are closely connected to the application and can enable direct attacks. These vulnerabilities generally arise from poor server configuration or security flaws in the web server software.

Vulnerable Web Server Configuration:

Default Credentials:

Many web servers have publicly accessible administrative interfaces on the web root or ports like 8080 or 8443. These interfaces often use default usernames and passwords that administrators fail to change. Attackers can find these interfaces using port scanning and try known default credentials. If successful, they can gain admin access and compromise the server and application.

	USERNAME	PASSWORD
Apache Tomcat	admin	(none)
	tomcat	tomcat
	root	root
Sun JavaServer	admin	admin
Netscape Enterprise Server	admin	admin
Compaq Insight Manager	administrator	administrator
	anonymous	(none)
	user	user
	operator	operator
	user	public
Zeus	admin	(none)

Default Content:

Most web servers include default content and features that can be misused by attackers. This may include debug or test functions, sample applications, and powerful tools not meant for public access. Sometimes server manuals and documentation are also left accessible, revealing useful system details. Attackers can use this information or functionality to help compromise the server or web application.

➔ **Debug Functionality:**

Debug functionality created for administrators can expose sensitive information to attackers. These features may reveal details about the server configuration, environment, and running applications. For example, the `phpinfo.php` page in many Apache servers runs the `phpinfo()` function and displays detailed PHP configuration information. Attackers can use this data to understand the system and find potential vulnerabilities.

➔ **Sample Functionality:**

Many web servers include sample scripts and pages to demonstrate how server functions work. Sometimes these scripts contain security vulnerabilities that attackers can exploit. In other cases, they provide useful functionality, such as viewing source code or modifying session data. Attackers can use these weaknesses to access sensitive information or manipulate the web application.

➔ **Powerful Functions:**

Some web servers contain powerful built-in functions that are not meant for public use but may still be accessible. For example, the PL/SQL gateway in Oracle Application Server allows web requests to run database procedures through URLs. Attackers can exploit this to execute database queries and retrieve sensitive data. Although security filters like the PL/SQL Exclusion List try to block dangerous procedures, incomplete filtering may still allow attackers to access powerful database functions.

Directory Listings:

When a web server receives a request for a directory instead of a file, it may return a default page, show an error (like HTTP 403), or display a directory listing of files. Directory listings can reveal files such as logs, backups, or old scripts that should not be publicly accessible. Attackers can use this information to discover sensitive resources and application structure. This can help them exploit other vulnerabilities and potentially compromise the application.

Dangerous HTTP Methods:

Some web servers allow dangerous HTTP methods beyond the common GET and POST. Methods like PUT, DELETE, COPY, MOVE, and TRACE can be misused by attackers to upload files, delete resources, or gather sensitive information. The PUT method is especially risky because it can allow attackers to upload malicious scripts to the server. Attackers can use the OPTIONS method to check which HTTP methods are enabled. If these methods are improperly configured, they can lead to serious security vulnerabilities.

The Web Server as a Proxy:

A web server may sometimes be configured as a forward or reverse proxy. If a forward proxy is misconfigured, attackers can misuse it to send attacks to other systems, making the traffic appear as if it comes from the proxy server. They may also use it to access internal network systems that are normally not reachable from the internet. Methods like GET with full URLs or CONNECT requests can force the proxy to connect to other hosts. This can allow attackers to scan internal networks, access services, or bypass security controls.

Misconfigured Virtual Hosting:

Misconfigured virtual hosting occurs when a web server hosts multiple websites using the HTTP Host header, but the default host is not properly secured. Security settings may only apply to a specific virtual host while the default host remains unprotected. Attackers can bypass restrictions by sending requests with a different, missing, or IP-based Host header. This may expose directory listings or different default content, helping attackers discover sensitive resources.

Securing Web Server Configuration:

To secure a web server configuration, administrators should remove common weaknesses and follow proper hardening practices. This includes changing default credentials, blocking public access to administrative interfaces, and removing unnecessary default content or features. Directory listings should be disabled, and only required HTTP methods like GET and POST should be allowed. The server should not act as a proxy unless necessary, and virtual hosting must be configured securely, including protection for the default host. These steps help reduce the risk of attackers exploiting server misconfigurations.

Vulnerable Web Server Software:

Web server software ranges from simple tools that serve static pages to complex platforms handling many tasks, and has historically suffered from serious vulnerabilities like arbitrary code execution, file disclosure, and privilege escalation. Rather than focusing on patched vulnerabilities, which quickly become outdated, it is more important to understand the underlying principles and techniques behind them. By studying common defect types and using systematic methods, new vulnerabilities—such as directory listing and source code disclosure—can be effectively identified.

Buffer Overflow Vulnerabilities:

Buffer overflow vulnerabilities are among the most serious software flaws, as they can allow attackers to gain control over a process and execute arbitrary code. In the context of web servers, this can lead to full compromise of hosted applications. These vulnerabilities are widespread and have affected many different web server products and components, highlighting their persistent and critical nature.

Several major web server products have suffered from buffer overflow vulnerabilities that allowed attackers to execute arbitrary code and fully compromise systems. Examples include Microsoft IIS ISAPI extensions (like Internet Printing Protocol and Index Server), which enabled worm propagation such as Nimda and Code Red, as well as flaws in Apache's chunked encoding and IIS WebDAV components. Similar issues were also found in the iPlanet search component, highlighting how widespread and critical buffer overflow flaws have been across different web server technologies.

Path Traversal Vulnerabilities:

Path traversal vulnerabilities in web server software allow attackers to access or modify files outside the web root by manipulating file paths in requests. These flaws have appeared in multiple products, such as Accipiter DirectServer, Alibaba, Cisco ACS Acme.server, and McAfee EPolicy Orchestrator, often using techniques like dot-dot slash sequences or crafted URLs. Such vulnerabilities can lead to unauthorized file access or writing anywhere on the system, posing serious security risks.

Encoding and Canonicalization Vulnerabilities:

Encoding and canonicalization vulnerabilities arise when different system components process encoded data inconsistently, allowing attackers to bypass filters. These flaws can lead to issues like directory listing, path traversal, and command execution. They highlight the risk of mismatched input validation across multiple layers of a web system.

Finding Web Server Flaws:

Finding web server flaws often involves using automated scanning tools like Nessus to detect known vulnerabilities in widely used third-party software. However, since servers are usually patched, additional research through sources like SecurityFocus and Bugtraq is essential to identify recent or undisclosed issues. Manual testing and local installation of the target software can further help uncover hidden or new vulnerabilities that automated tools might miss.

Securing Web Server Software:

Securing web server software involves choosing reliable products with a strong security track record and applying vendor patches promptly to fix known vulnerabilities. Organizations should also harden servers by disabling unnecessary features, renaming defaults, and enforcing least privilege to reduce attack surfaces.

Additionally, continuous monitoring for new vulnerabilities and implementing defense-in-depth (e.g., network filters and intrusion detection) help minimize damage and respond quickly to potential breaches.

--The End--